Structures in C Programming

30 May 2026 20:08

A **structure(struct)** in C is a user-defined data type that allows grouping variables of **different data types** under a single name.

Structures are used when we want to represent a real-world entity such as:

- Student
- Employee
- Book
- Product
- Ordered Pair
- Player
- Bank Account
- Etc.

Each entity contains multiple attributes of different data types.

Why use Structures?

Without structures:

```
char name[50];
int roll;
float marks;
```

Managing multiple students becomes difficult.

Using structures:

```
struct Student{
   int roll;
   char name[50];
   float marks;
};
```

Now all student-related data is grouped together.

Syntax of structure:

```
struct structure_name{
   data_type variable1;
   data_type variable2;
   …
   data_type variablen;
};
```

For example:

```
struct Student{
```

```
    int roll;
    char name[50];
    float marks;
};
```

Declaring Structure Variables:
After defining a structure, create variables of that type.

```
struct Student s1, s2;
```

For example:

```
struct Student{
    int roll;
    char name[50];
    float marks;
};

int main(){
    struct Student s1;
}
```

Accessing Structure Members:
Use the **dot(.) operator**

Syntax:

```
structure_variable.member_name;
```

For example:

```
s1.roll = 101;
s1.marks = 95.7;
```

```
#include<stdio.h>
struct Student{
    int rollNum;
    char name[50];
    float marks;
};
int main1(int argc, char const *argv[])
{
    struct Student s1;
    printf("Enter roll number: ");
    scanf("%d", &s1.rollNum);
    fflush(stdin);
```

```
        printf("Enter student name: ");
        fgets(s1.name, 50, stdin);
        printf("Enter marks: ");
        scanf("%f", &s1.marks);
        //display the information
        printf("\nRoll number: %d", s1.rollNum);
        printf("\nName: %s", s1.name);
        printf("Marks: %.2f", s1.marks);
        return 0;
    }
```

Output:

```
Enter roll number: 1
Enter student name: Raju
Enter marks: 67.8

Roll number: 1
Name: Raju
Marks: 67.80
```

Example: 2

```
    #include<stdio.h>
    #include"one.C"
    #define SIZE 5
    int main(int argc, char const *argv[])
    {
        struct Student stu[SIZE];
        int i;
        printf("Enter the details of %d students\n", SIZE);
        for(i = 0; i < SIZE; i++){
            printf("Enter the roll number: ");
            scanf("%d", &stu[i].rollNum);
            fflush(stdin);
            printf("Enter student name: ");
            fgets(stu[i].name, 50, stdin);
            printf("Enter marks: ");
            scanf("%f", &stu[i].marks );
        }
        printf("\nStudents details:\n");
        for(i = 0; i < SIZE; i++){
            printf("\nDetails of student: %d\n",(i+1));
            printf("Roll number: %d\n", stu[i].rollNum);
            printf("Name: %s", stu[i].name);
            printf("Marks: %.2f", stu[i].marks);
```

```
        }
        return 0;
    }
```

Output:

```
    Enter the details of 5 students
    Enter the roll number: 1
    Enter student name: Soumo
    Enter marks: 88.5
    Enter the roll number: 2
    Enter student name: Sapta
    Enter marks: 90.33
    Enter the roll number: 3
    Enter student name: Shubho
    Enter marks: 77.6
    Enter the roll number: 4
    Enter student name: Yash
    Enter marks: 89
    Enter the roll number: 5
    Enter student name: Pankaj
    Enter marks: 78.678

    Students details:

    Details of student: 1
    Roll number: 1
    Name: Soumo
    Marks: 88.50
    Details of student: 2
    Roll number: 2
    Name: Sapta
    Marks: 90.33
    Details of student: 3
    Roll number: 3
    Name: Shubho
    Marks: 77.60
    Details of student: 4
    Roll number: 4
    Name: Yash
    Marks: 89.00
    Details of student: 5
    Roll number: 5
    Name: Pankaj
    Marks: 78.68
```

Structure initialization

Members can be initialized during declaration.

```
#include<stdio.h>
struct Student{
    int rollNum;
    char name[50];
    float marks;
};
int main(int argc, char const *argv[])
{
    struct Student s1 = {1, "Shiv\n", 88.88};

    //display the information
    printf("\nRoll number: %d", s1.rollNum);
    printf("\nName: %s", s1.name);
    printf("Marks: %.2f", s1.marks);
    return 0;
}
```

Output:

```
Roll number: 1
Name: Shiv
Marks: 88.88
```

Nested Structure

A structure inside another structure.

Example:

```
#include <stdio.h>
struct Date
{
    int day;
    int month;
    int year;
};
struct Student
{
    int rollNum;
    char name[50];
    float marks;
    struct Date dob;
```

```c
};
int main(int argc, char const *argv[])
{
    struct Student s1;
    printf("Enter roll number: ");
    scanf("%d", &s1.rollNum);
    fflush(stdin);
    printf("Enter student name: ");
    fgets(s1.name, 50, stdin);
    printf("Enter marks: ");
    scanf("%f", &s1.marks);
    printf("Enter DOB (dd mm yyyy): ");
    scanf("%d%d%d", &s1.dob.day, &s1.dob.month, &s1.dob.year);
    // display the information
    printf("\nRoll number: %d", s1.rollNum);
    printf("\nName: %s", s1.name);
    printf("Marks: %.2f", s1.marks);
    printf("\nDOB: %d/%d/%d", s1.dob.day, s1.dob.month, s1.dob.year);
    return 0;
}
```

Output:

```
Enter roll number: 1
Enter student name: Bhem
Enter marks: 85.9
Enter DOB (dd mm yyyy): 12 11 1991

Roll number: 1
Name: Bhem
Marks: 85.90
DOB: 12/11/1991
```

Structure and Functions

Structures can be passes to functions.

Passing structure as Argument

```c
#include <stdio.h>
struct Date
{
    int day;
    int month;
    int year;
```

```
};
struct Student
{
    int rollNum;
    char name[50];
    float marks;
    struct Date dob;
};
void display(struct Student st){
    // display the information
    printf("\nRoll number: %d", st.rollNum);
    printf("\nName: %s", st.name);
    printf("Marks: %.2f", st.marks);
    printf("\nDOB: %d/%d/%d", st.dob.day, st.dob.month, st.dob.year);
}
int main(int argc, char const *argv[])
{
    struct Student s1;
    printf("Enter roll number: ");
    scanf("%d", &s1.rollNum);
    fflush(stdin);
    printf("Enter student name: ");
    fgets(s1.name, 50, stdin);
    printf("Enter marks: ");
    scanf("%f", &s1.marks);
    printf("Enter DOB (dd mm yyyy): ");
    scanf("%d%d%d", &s1.dob.day, &s1.dob.month, &s1.dob.year);

    display(s1);
    return 0;
}
```

Output:

```
Enter roll number: 1
Enter student name: Jai
Enter marks: 66.812
Enter DOB (dd mm yyyy): 15 12 1990

Roll number: 1
Name: Jai
Marks: 66.81
DOB: 15/12/1990
```

Returning Structure from Function

```c
#include <stdio.h>
struct Date
{
    int day;
    int month;
    int year;
};
struct Student
{
    int rollNum;
    char name[50];
    float marks;
    struct Date dob;
};
void display(struct Student st){
    // display the information
    printf("\nRoll number: %d", st.rollNum);
    printf("\nName: %s", st.name);
    printf("Marks: %.2f", st.marks);
    printf("\nDOB: %d/%d/%d", st.dob.day, st.dob.month, st.dob.year);
}
struct Student createStudent(){ //function returning structure
    struct Student s1;
    printf("Enter roll number: ");
    scanf("%d", &s1.rollNum);
    fflush(stdin);
    printf("Enter student name: ");
    fgets(s1.name, 50, stdin);
    printf("Enter marks: ");
    scanf("%f", &s1.marks);
    printf("Enter DOB (dd mm yyyy): ");
    scanf("%d%d%d", &s1.dob.day, &s1.dob.month, &s1.dob.year);

    return s1;
}
int main(int argc, char const *argv[])
{
    struct Student stu;
    stu = createStudent();
    display(stu);
    return 0;
}
```

Output:

```
Enter roll number: 1
Enter student name: Harsh
Enter marks: 67.234
Enter DOB (dd mm yyyy): 16 7 1995

Roll number: 1
Name: Harsh
Marks: 67.23
DOB: 16/7/1995
```

Pointer to Structure

A pointer can store the address of a Structure.

```
struct Student s1;
struct Student *ptr;

ptr = &s1;
```

Accessing members using pointer

Use the arrow operator(->)

```
ptr->rollNum
ptr->marks
```

Equivalent:

```
(*ptr).rollNum
(*ptr).marks
```

```c
#include <stdio.h>
struct Date
{
    int day;
    int month;
    int year;
};
struct Student
{
    int rollNum;
    char name[50];
    float marks;
    struct Date dob;
};
void display(struct Student *st){
    // display the information
```

```c
    printf("\nRoll number: %d", st->rollNum);
    printf("\nName: %s", st->name);
    printf("Marks: %.2f", st->marks);
    printf("\nDOB: %d/%d/%d", st->dob.day, st->dob.month, st->dob.year);
}
struct Student createStudent(){
    struct Student s1;
    printf("Enter roll number: ");
    scanf("%d", &s1.rollNum);
    fflush(stdin);
    printf("Enter student name: ");
    fgets(s1.name, 50, stdin);
    printf("Enter marks: ");
    scanf("%f", &s1.marks);
    printf("Enter DOB (dd mm yyyy): ");
    scanf("%d%d%d", &s1.dob.day, &s1.dob.month, &s1.dob.year);

    return s1;
}
int main(int argc, char const *argv[])
{
    struct Student stu;
    stu = createStudent();
    struct Student *ptr;
    ptr = &stu;
    display(ptr);
    return 0;
}
```

```
Output:
   Enter roll number: 1
   Enter student name: Harsh
   Enter marks: 56.99
   Enter DOB (dd mm yyyy): 12 7 1999

   Roll number: 1
   Name: Harsh
   Marks: 56.99
   DOB: 12/7/1999
```

typedef with Structures

typedef creates an alias.

```c
#include <stdio.h>
```

```c
struct Date
{
    int day;
    int month;
    int year;
};
typedef struct
{
    int rollNum;
    char name[50];
    float marks;
    struct Date dob;
}  Student;
void display(Student *st){
    // display the information
    printf("\nRoll number: %d", st->rollNum);
    printf("\nName: %s", st->name);
    printf("Marks: %.2f", st->marks);
    printf("\nDOB: %d/%d/%d", st->dob.day, st->dob.month, st->dob.year);
}
Student createStudent(){
    Student s1;
    printf("Enter roll number: ");
    scanf("%d", &s1.rollNum);
    fflush(stdin);
    printf("Enter student name: ");
    fgets(s1.name, 50, stdin);
    printf("Enter marks: ");
    scanf("%f", &s1.marks);
    printf("Enter DOB (dd mm yyyy): ");
    scanf("%d%d%d", &s1.dob.day, &s1.dob.month, &s1.dob.year);

    return s1;
}
int main(int argc, char const *argv[])
{
    Student stu;
    stu = createStudent();
    Student *ptr;
    ptr = &stu;
    display(ptr);
    return 0;
}
```

Output:

```
Enter roll number: 1
Enter student name: Jai
Enter marks: 88.78
Enter DOB (dd mm yyyy): 17 11 1999

Roll number: 1
Name: Jai
Marks: 88.78
DOB: 17/11/1999
```

Structured Size

Use sizeof():

```
printf("\nSize of structure in bytes: %lu", sizeof(stu));
```

Output:

Size of structure in bytes: **72 (depends upon compiler and padding)**

Structure Padding

Compiler adds extra bytes to improve memory alignment.